

Análisis de la clase ProbNet
Defectos de diseño, bugs y propuesta de rediseño
org.openmarkov.core — paquete model.network

Manuel Arias Calleja

Marzo de 2026

Índice

1. Introducción	3
2. Fundamentos teóricos	3
2.1. Principio de Responsabilidad Única (SRP)	3
2.2. Patrón <i>Extract Class</i>	3
2.3. Antipatrón <i>God Class</i>	3
2.4. Principio DRY	3
2.5. Acoplamiento	4
3. Situación actual	4
4. Bugs detectados	5
4.1. B1: <code>getChanceAndDecisionVariables()</code> incluye tipos extra	5
4.2. B2: <code>moveNode()</code> — <code>NullPointerException</code> potencial	5
4.3. B3: <code>modifyAgent(REMOVE)</code> deja <code>agents = null</code>	5
4.4. B4: <code>addShiftedNode()</code> no copia potenciales	5
5. Code smells	6
5.1. S1: Lógica duplicada en <code>checkProbNet()</code> y <code>getUnsatisfiedConstraints()</code> (violación DRY)	6
5.2. S2: Bucles manuales en clasificación de red	6
5.3. S3: Patrón duplicado en <code>getProbPotentials()/getUtilityPotentials()</code> (violación DRY)	6
5.4. S4: <code>getNumCriteria()</code> con complejidad $O(n \cdot m)$	6
5.5. S5: Código comentado en <code>addNodeConsistently()</code>	6
5.6. S6: <code>toString()</code> en la línea 53	6
5.7. S7: <code>reader/writer</code> almacenados en ProbNet (violación SRP y DIP)	7
6. Cobertura de tests	7
6.1. Nodos, links y variables	8
6.2. Potenciales, constraints y copia	9
6.3. Clasificación, agentes e I/O	10
6.4. Resumen cuantitativo	10
7. Propuesta de rediseño	11
7.1. Fase 0: Ampliar tests (prerrequisito)	12
7.2. Fase 1: Corregir bugs (sin cambio de API)	12

7.3.	Fase 2: Simplificaciones internas (sin cambio de API)	12
7.4.	Fase 3: Extraer <code>ProbNetClassifier</code> (<i>Extract Class</i>)	12
7.5.	Fase 4: Extraer <code>ProbNetAgentManager</code> (<i>Extract Class</i>)	13
7.6.	Fase 5: Extraer <code>ProbNetPotentialQueries</code> (<i>Extract Class</i>)	13
7.7.	Fase 6: Evaluar extracción de reader/writer (reducir acoplamiento)	13
8.	Resumen de impacto estimado	13
9.	Orden de ejecución y riesgos	13

1. Introducción

ProbNet es la clase más crítica de OpenMarkov: **490 ficheros** la referencian en todo el proyecto. Representa una red probabilista (bayesiana, de influencia, dinámica, etc.) y actúa como contenedor central de nodos, arcos, potenciales, restricciones y metadatos.

Con **1440 líneas** y **80+ métodos públicos**, la clase ha acumulado responsabilidades que van más allá de su propósito original. Este documento presenta un análisis detallado de la situación actual, identifica bugs, code smells y lagunas de testing, y propone una hoja de ruta de rediseño incremental.

2. Fundamentos teóricos

Esta sección resume los principios y patrones de ingeniería del software que se aplican —o que se han detectado como violados— en el análisis de **ProbNet**.

2.1. Principio de Responsabilidad Única (SRP)

El *Single Responsibility Principle* [2] establece que **una clase debe tener una única razón para cambiar**. Si una clase acumula múltiples responsabilidades, un cambio en una de ellas puede introducir errores en las demás.

ProbNet tiene al menos seis áreas de responsabilidad distintas (nodos, potenciales, constraints, clasificación, agentes, I/O), lo que viola directamente este principio.

2.2. Patrón *Extract Class*

Extract Class [1] es la técnica que se aplica cuando una clase tiene demasiadas responsabilidades: se identifica un grupo cohesivo de métodos, se mueve a una nueva clase y se sustituye en la clase original por una delegación.

En OpenMarkov, este patrón ya se ha aplicado con éxito: primero con **VariableTypeConverter** (extraída de **Node**) y después con **VariableStateOperations** (extraída de **Variable**). La propuesta sigue el mismo precedente.

2.3. Antipatrón *God Class*

Una *God Class* (también llamada *Blob* o *Monster Class*) es una clase que sabe demasiado o hace demasiado [1]. Se reconoce por:

- Cientos o miles de líneas.
- Métodos que no se relacionan entre sí.
- Muchas dependencias entrantes (otros módulos la necesitan para todo).

Con 1440 líneas, 80+ métodos y 490 ficheros dependientes, **ProbNet** exhibe todas las características de una *God Class*. El objetivo del rediseño es reducirla a su rol esencial de **contenedor de la red probabilística**, delegando las responsabilidades secundarias.

2.4. Principio DRY

Don't Repeat Yourself [3] establece que cada pieza de conocimiento debe tener una única representación en el sistema. Hemos detectado varias violaciones de DRY en **ProbNet**: **checkProbNet()** duplica la lógica de **getUnsatisfiedConstraints()**, y **getProbPotentials()** repite el patrón de **getUtilityPotentials()**.

2.5. Acoplamiento

Decimos que una clase A está **acoplada** a otra B cuando A necesita conocer B para funcionar. ProbNet almacena campos de tipo ProbNetReader y ProbNetWriter, acoplando el modelo de dominio al sistema de entrada/salida. El principio de *Inversión de Dependencias* (DIP) sugiere que las clases de alto nivel (modelo) no deben depender de clases de bajo nivel (I/O).

3. Situación actual

La Figura 1 muestra un diagrama UML de ProbNet con sus métodos agrupados por responsabilidad. Las clases asociadas se disponen en vertical debajo de ProbNet.

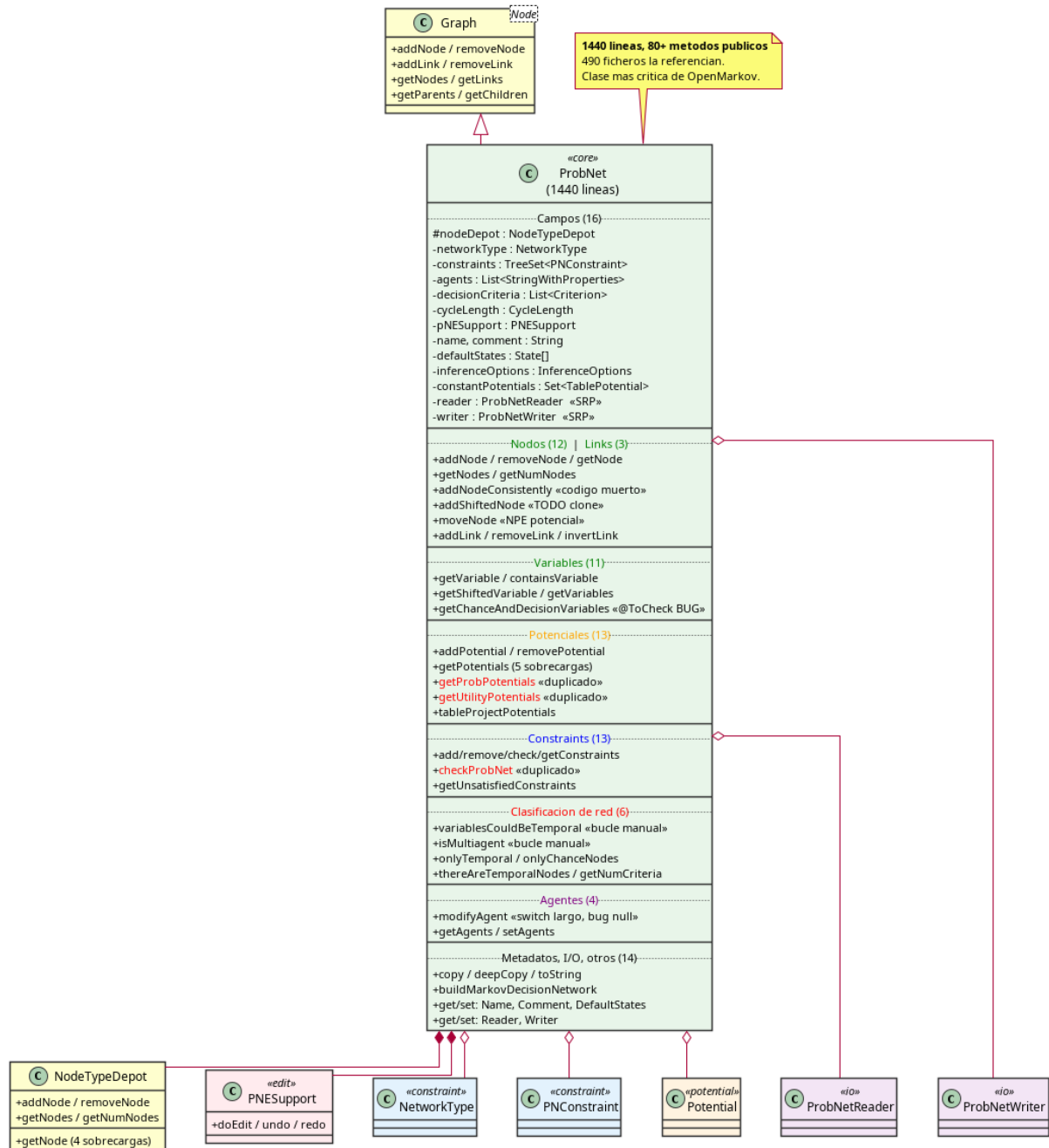


Figura 1: Diagrama UML de ProbNet (1440 líneas). Los colores indican áreas de responsabilidad; las etiquetas en rojo señalan problemas.

ProbNet extiende `Graph<Node>` (grafo genérico) y añade:

- **16 campos:** desde `nodeDepot` (indexación de nodos por tipo) hasta `reader/writer` (I/O).
- **80+ métodos públicos:** gestión de nodos (12), links (3), variables (11), potenciales (13), constraints (13), clasificación de red (6), agentes (4), metadatos e I/O (12), y otros (copia, inferencia, criterios).

4. Bugs detectados

La Figura 2 resume los bugs, code smells y riesgos de tipo identificados.

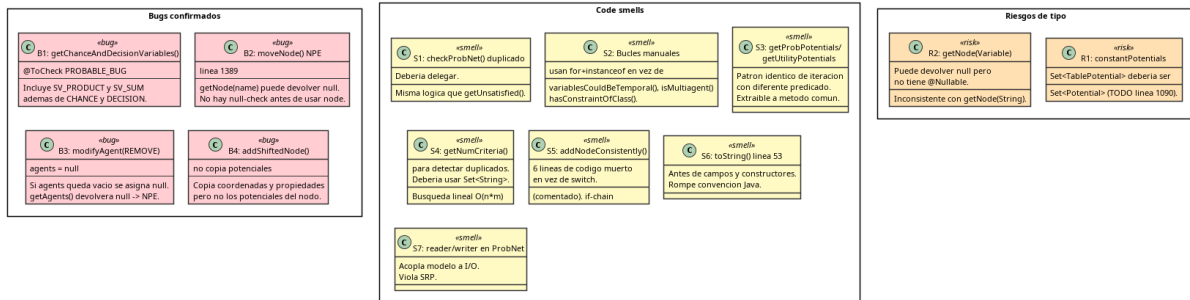


Figura 2: Bugs confirmados (rojo), code smells (amarillo) y riesgos de tipo (naranja) detectados en ProbNet.

4.1. B1: getChanceAndDecisionVariables() incluye tipos extra

El método está anotado con `@ToCheck(PROBABLE_BUG)`. Su nombre indica que devuelve solo variables de tipo `CHANCE` y `DECISION`, pero el `switch` también incluye `SV_PRODUCT` y `SV_SUM`:

```
1 case CHANCE, SV_PRODUCT, SV_SUM, DECISION -> true;
2 case UTILITY -> false;
```

Si el comportamiento actual es intencionado, el nombre del método es engañoso. Si no lo es, devuelve variables de más.

4.2. B2: moveNode() — NullPointerException potencial

En la línea 1389, `getNode(name)` puede devolver `null` si el nombre no existe en la red. No hay comprobación de nulidad antes de llamar a `node.setCoordinateX()`.

```
1 Node node = getNode(name);
2 node.setCoordinateX(newPositions.get(i).getX()); // NPE si node == null
```

4.3. B3: modifyAgent(REMOVE) deja agents = null

Cuando se elimina el último agente, el método asigna `agents = null` (línea 1426). Sin embargo, `getAgents()` devuelve directamente el campo sin comprobación, lo que causa un `NullPointerException` en cualquier código que itere sobre el resultado. Además, `modifyAgent(ADD)` crea la lista si es `null`, pero nunca debería haberlo sido.

4.4. B4: addShiftedNode() no copia potenciales

El método copia el tipo de nodo, coordenadas, propósito, relevancia, comentario y propiedades adicionales, pero **no copia los potenciales** del nodo original ni crea uno por defecto. El propio

código reconoce el problema:

```
1 // TODO Hacer clon para node y quitar estas lineas
```

5. Code smells

5.1. S1: Lógica duplicada en `checkProbNet()` y `getUnsatisfiedConstraints()` (violación DRY)

Ambos métodos iteran sobre `constraints` con la misma condición `constraint.isMetBy(this)`. La solución trivial:

```
1 public boolean checkProbNet() {
2     return getUnsatisfiedConstraints().isEmpty();
3 }
```

5.2. S2: Bucles manuales en clasificación de red

`variablesCouldBeTemporal()` e `isMultiagent()` usan un bucle `for` manual sobre `constraints` con `instanceof`, cuando ya existe `hasConstraintOfClass()` que hace exactamente eso:

```
1 // Antes (7 lineas):
2 public boolean variablesCouldBeTemporal() {
3     for (PNConstraint constraint : constraints) {
4         if (constraint instanceof OnlyAtemporalVariables) return false;
5     }
6     return true;
7 }
8
9 // Despues (1 linea):
10 public boolean variablesCouldBeTemporal() {
11     return !hasConstraintOfClass(OnlyAtemporalVariables.class);
12 }
```

5.3. S3: Patrón duplicado en `getProbPotentials()`/`getUtilityPotentials()` (violación DRY)

Ambos métodos siguen el mismo patrón: obtener el nodo de la variable, recopilar vecinos, iterar potenciales y filtrar. Solo difiere el predicado de filtro. Se podrían unificar en un método privado parametrizado o extraer a una clase de consultas.

5.4. S4: `getNumCriteria()` con complejidad $O(n \cdot m)$

Usa una `List<String>` y búsqueda lineal para detectar duplicados en lugar de un `Set<String>`, resultando en complejidad cuadrática innecesaria.

5.5. S5: Código comentado en `addNodeConsistently()`

Las líneas 933–938 contienen 6 líneas de código comentado que nunca se ejecutan. Además, usa una cadena de `if` en lugar de `switch`.

5.6. S6: `toString()` en la línea 53

El método `toString()` está al inicio del fichero, antes incluso de la declaración de campos. Rompe la estructura convencional de Java (campos $\rightarrow$ constructores $\rightarrow$ métodos).

5.7. S7: reader/writer almacenados en ProbNet (violación SRP y DIP)

`ProbNet` almacena referencias a `ProbNetReader` y `ProbNetWriter`, acoplando el modelo de dominio al sistema de I/O. El reader/writer solo se usa en la GUI para recordar qué formato se usó. Esta información debería residir en el controlador de ficheros, no en el modelo de dominio.

6. Cobertura de tests

Las siguientes figuras muestran el estado de la cobertura de tests por área funcional. El código de colores: **verde** = testado, **amarillo** = parcialmente testado, **rojo** = sin tests.

6.1. Nodos, links y variables

C «tested» Gestion de nodos (64%) [OK] addNode(Variable, NodeType) [OK] addNode(Node) [OK] removeNode(Node) [OK] getNode(String) [OK] getNode(String, NodeType) [OK] getNodes() [OK] getNodes(NodeType) [OK] getNodes(List<Variable>) [OK] getNumNodes() / getNumNodes(NodeType) [NO] addNodeConsistently() [NO] addShiftedNode() [NO] moveNode() [NO] getNode(Variable) [NO] getNodes(CECriterion)	C «partial» Gestion de links (67%) [OK] addLink(Var, Var, bool) [OK] removeLink(Var, Var, bool) [NO] invertLink() [NO] links no dirigidos
C «partial» Variables (36%) [OK] getVariables() [OK] getVariables(NodeType) [OK] getChanceAndDecisionVariables() [OK] static getVariables(nodes) [NO] getVariable(String) [NO] getVariable(baseName, timeSlice) [NO] containsVariable() (2 sobrec.) [NO] containsShiftedVariable() [NO] getShiftedVariable() [NO] updateVariable() [NO] getVariablesNames()	

Figura 3: Cobertura de tests: nodos (64 %), links (67 %) y variables (36 %).

6.2. Potenciales, constraints y copia

C «<i>partial</i>» Potenciales (56%) [OK] addPotential(Potential) [OK] removePotential(Potential) [OK] removePotentials(Node) [OK] getPotentials() [OK] getPotentials(Variable) [OK] getPotentialsByType(NodeType) [OK] getUtilityPotentials(Variable) [OK] getNumPotentials() [OK] tableProjectPotentials(EvidenceCase) [NO] getPotentials(Predicate) [NO] getPotentialsByRole(PotentialRole) [NO] getAdditivePotentials() [NO] getProbPotentials(Variable) [NO] getSortedPotentials() [NO] removePotentials(List) [NO] getConstantPotentials() [NO] addPotential(Potential, ProbNet)	C «<i>partial</i>» Constraints (31%) [OK] addConstraint(PNConstraint) [OK] removeConstraint(PNConstraint) [OK] getConstraints() [OK] getAdditionalConstraints() [NO] checkConstraints() [NO] checkConstraints(Iterable) [NO] setNetworkType(NetworkType) [NO] checkProbNet() [NO] getUnsatisfiedConstraints() [NO] hasConstraintOfClass() [NO] getConstraintOfClass() [NO] getConstraintsOfClass() [NO] removeAllConstraints()
C «<i>tested</i>» Copiar (100%) [OK] copy() [OK] deepCopy() (30 tests en ProbNetCopyTest)	

Figura 4: Cobertura de tests: potenciales (56 %), constraints (31 %) y copia (100 %).

6.3. Clasificación, agentes e I/O

<i>«untested»</i> Clasificación de red (0%) [NO] variablesCouldBeTemporal() [NO] isMultiagent() [NO] onlyTemporal() [NO] onlyChanceNodes() [NO] thereAreTemporalNodes() [NO] getNumCriteria() 6 metodos sin ningun test. Incluye logica duplicada.	<i>«untested»</i> Agentes (0%) [NO] modifyAgent(StateAction, ...) [NO] getAgents() [NO] setAgents(List) modifyAgent() tiene un bug conocido y codigo complejo.
<i>«untested»</i> I/O y metadatos (0%) [NO] getReader / setReader [NO] getWriter / setWriter [NO] buildMarkovDecisionNetwork() [NO] getDefaultStates / setDefaultStates [NO] getComment / setComment [NO] getShowCommentWhenOpening / set... [NO] getName / setName 12 metodos sin ningun test.	

Figura 5: Cobertura de tests: clasificación de red (0 %), agentes (0 %) e I/O y metadatos (0 %). Estas tres áreas son las lagunas más críticas.

6.4. Resumen cuantitativo

Área	Métodos	Testados	Cobertura
Gestión de nodos	14	9	64 %
Gestión de links	3	2	67 %
Variables	11	4	36 %
Potenciales	16	9	56 %
Constraints	13	4	31 %
Clasificación de red	6	0	0 %
Agentes	4	0	0 %
Copiar	2	2	100 %
I/O y metadatos	12	0	0 %
Total	81	30	37 %

Existen **30 tests** en ProbNetTest y **30 tests** en ProbNetCopyTest, pero **51 métodos públicos** no tienen ningún test unitario directo.

7. Propuesta de rediseño

Dado que ProbNet es referenciada por 490 ficheros, toda refactorización debe ser **incremental** y **retrocompatible**. Se aplica el patrón *Extract Class* siguiendo el precedente de VariableTypeConverter y VariableStateOperations.

La Figura 6 muestra la arquitectura objetivo.

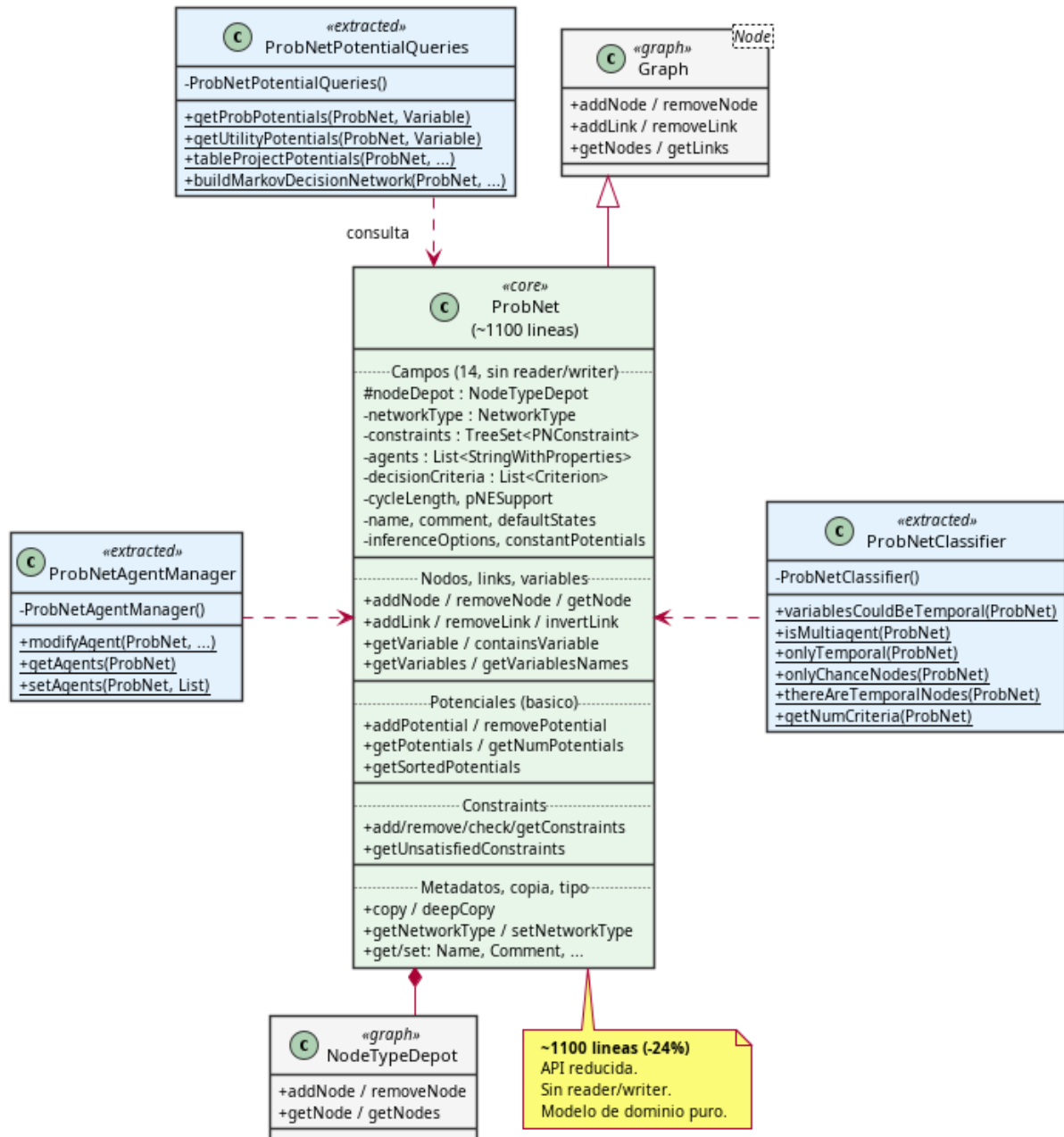


Figura 6: Arquitectura propuesta: ProbNet reducida a ~1100 líneas, con tres clases de utilidad estática extraídas.

7.1. Fase 0: Ampliar tests (prerrequisito)

Antes de cualquier refactorización, cubrir las lagunas críticas:

Área	Tests a añadir	Prioridad
Variables	<code>getVariable</code> , <code>containsVariable</code> , <code>getShiftedVariable</code> , <code>getVariablesNames</code> , <code>updateVariable</code>	Alta
Constraints	<code>setNetworkType</code> (cambio válido e inválido con roll-back), <code>checkProbNet</code> , <code>getUnsatisfiedConstraints</code> , <code>hasConstraintOfClass</code>	Alta
Clasificación	Todos los 6 métodos con redes BN, ID y DBN	Alta
Agentes	<code>modifyAgent</code> ADD/REMOVE/RENAME con casos límite	Alta
Potenciales	<code>addPotential(pot, origNet)</code> , <code>getProbPotentials</code> , <code>getSortedPotentials</code> , <code>getConstantPotentials</code>	Media
Nodos	<code>moveNode</code> (caso normal y nombre inválido), <code>addShiftedNode</code> , <code>invertLink</code>	Media
Metadatos	<code>getDefaultStates</code> , <code>comment</code> , <code>showCommentWhenOpening</code>	Baja

Objetivo: pasar de **37 %** a **>80 %** de cobertura de métodos públicos antes de empezar las extracciones.

7.2. Fase 1: Corregir bugs (sin cambio de API)

Bug	Corrección propuesta
B1	Renombrar a <code>getNonUtilityVariables()</code> o eliminar <code>SV_PRODUCT/SV_SUM</code> del switch según la intención original (requiere consultar el uso en el proyecto).
B2	Añadir null-check en <code>moveNode()</code> con <code>Objects.requireNonNull</code> o skip silencioso.
B3	Inicializar <code>agents</code> como lista vacía en lugar de <code>null</code> ; eliminar asignación <code>agents = null</code> en <code>modifyAgent(REMOVE)</code> .
B4	En <code>addShiftedNode()</code> , añadir copia de potenciales o crear potencial uniforme por defecto (verificar qué espera el código llamador).

7.3. Fase 2: Simplificaciones internas (sin cambio de API)

- `checkProbNet()` → delegar en `getUnsatisfiedConstraints().isEmpty()` (eliminar duplicación DRY).
- `variablesCouldBeTemporal()` e `isMultiagent()` → usar `hasConstraintOfClass()` (eliminar bucles manuales).
- `getNumCriteria()` → usar `Set<String>` (mejorar complejidad algorítmica).
- Eliminar código comentado en `addNodeConsistently()`, convertir if-chain a switch.
- Mover `toString()` después de los campos.
- Añadir `@Nullable` a `getNode(Variable)`.

7.4. Fase 3: Extraer ProbNetClassifier (*Extract Class*)

Mover los 6 métodos de clasificación de red (`variablesCouldBeTemporal`, `isMultiagent`, `onlyTemporal`, `onlyChanceNodes`, `thereAreTemporalNodes`, `getNumCriteria`) a una clase de utilidad estática.

Dejar delegaciones en `ProbNet` para retrocompatibilidad.

7.5. Fase 4: Extraer `ProbNetAgentManager` (*Extract Class*)

Mover `modifyAgent()`, `getAgents()`, `setAgents()` a una clase separada. La lógica de agentes es completamente independiente del resto de `ProbNet` y solo se usa desde la GUI.

7.6. Fase 5: Extraer `ProbNetPotentialQueries` (*Extract Class*)

Mover `getProbPotentials()`, `getUtilityPotentials()`, `tableProjectPotentials()` y `buildMarkovDecision` a una clase de consultas. Unificar los patrones duplicados de iteración sobre vecinos (eliminando la violación DRY).

7.7. Fase 6: Evaluar extracción de reader/writer (reducir acoplamiento)

Evaluar mover `reader/writer` fuera de `ProbNet` hacia el controlador de ficheros de la GUI. Esto reduciría el acoplamiento del modelo con el sistema de I/O (DIP). Requiere un análisis de impacto en los módulos `gui` e `io`.

8. Resumen de impacto estimado

Métrica	Actual	Objetivo
Líneas de <code>ProbNet.java</code>	1440	~1100
Métodos públicos en <code>ProbNet</code>	80+	~65
Bugs conocidos	4	0
Code smells	7	0
Cobertura de tests	37 %	>80 %
Clases de utilidad nuevas	0	3

9. Orden de ejecución y riesgos

Fase	Descripción	Riesgo	Dependencia
0	Ampliar tests	Bajo	—
1	Corregir bugs	Bajo	Fase 0
2	Simplificaciones	Bajo	Fase 0
3	Extraer Classifier	Medio	Fase 0, 2
4	Extraer AgentManager	Bajo	Fase 0, 1
5	Extraer PotentialQueries	Medio	Fase 0
6	Evaluar reader/writer	Alto	Fase 0

Las fases 0–2 son de bajo riesgo y alto valor. Las fases 3–5 siguen el patrón *Extract Class* ya probado con éxito en `VariableStateOperations` y `VariableTypeConverter`. La fase 6 requiere un análisis adicional del módulo GUI.

Principio rector: cada fase debe dejar todos los tests pasando y no romper la API pública existente. Los métodos extraídos se delegan desde `ProbNet` hasta que los llamadores puedan migrar.

Referencias

- [1] M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed., Addison-Wesley, 2019.
- [2] R.C. Martin, *Agile Software Development: Principles, Patterns, and Practices*, Prentice Hall, 2003.
- [3] A. Hunt, D. Thomas, *The Pragmatic Programmer*, Addison-Wesley, 1999.